

📘 08차시 — SFTTrainer를 이용한 지도 미세조정

교육시간: 2일차 두 번째 시간 (60분)
검증 기준일: 2026-11-04

01 | 표지·목표·60분 진행

08차시 — SFTTrainer를 이용한 지도 미세조정. 2일차 두 번째 시간(60분). 07차시에서 정제한 train.jsonl과 validation.jsonl을 사용한다.

0~10분: 지도 미세조정 목표 / 10~20분: 데이터와 손실함수 / 20~30분: 설치·사전검사 / 30~45분: 5스텝 학습 / 45~55분: 평가·로그 / 55~60분: 체크리스트.

산출물: 학습 코드, 데이터 검증 로그, 학습 출력 디렉터리(실행 시). 실습 모델 Qwen/Qwen2.5-0.5B-Instruct는 교안 제작 선택이며 원본 과정의 필수 모델은 아니다.

---

02 | 배경·선수지식

SFT(Supervised Fine-Tuning, 지도 미세조정)는 입력과 모범 응답을 이용해 다음 토큰 예측 오차를 줄이는 학습이다. 02차시 토큰·트랜스포머, 04차시 LoRA, 07차시 JSONL 정제 개념을 연결한다.

이번 08차시는 SFTTrainer 자체를 이해하기 위해 전체 파라미터 학습을 짧게 수행한다. 09차시에서 LoRA를 결합하고 10차시에서 4비트 QLoRA를 결합한다. 전체 학습은 LoRA보다 GPU 메모리가 많이 필요하다.

학습 데이터는 정답 예시, 검증 데이터는 학습 중 일반화 확인에 사용한다. 예제 4+1개는 API 확인용이며 품질 향상을 입증할 만큼 충분하지 않다.

03 | 핵심 원리·손실과 스텝

토큰 시퀀스 x1, x2, ⋯, xn에 대해 모델은 앞선 토큰을 보고 다음 토큰의 확률을 예측한다. 교차 엔트로피 손실은 정답 토큰에 부여한 확률이 낮을수록 커진다.

배치 1, gradient_accumulation_steps 2는 두 미니배치의 기울기를 누적한 후 최적화 갱신을 수행한다. max_steps 5는 최적화 갱신 횟수를 5회로 제한한다. 따라서 예제 데이터가 작더라도 반복 순회할 수 있다.

training_loss는 학습 데이터의 목적함수 값이고 eval_loss는 별도 검증 데이터의 손실이다. 수치가 감소해도 사실성·안전성·정답률 개선을 보장하지 않는다.

---

04 | 전체 구조도·입출력

07차시 전처리 → processed/train.jsonl, validation.jsonl → JSONL 형식 검사 → datasets.Dataset → SFTTrainer → 토크나이저·채팅 템플릿 → 모델 순전파 → 손실 계산 → 역전파·가중치 갱신 → 평가 → outputs/lesson08_sft.

데이터는 messages 목록 안에 role과 content를 둔다. 예: user 질문 다음 assistant 모범답변. SFTTrainer는 대화형 데이터셋을 지원하며 처리 시 모델 채팅 템플릿을 사용한다.

주의: assistant_only_loss=True는 특정 채팅 템플릿에서 생성 구간 마스크를 지원해야 한다. 이 차시는 호환성 우선으로 False를 명시한다. 따라서 질문 토큰까지 손실에 포함될 수 있으며, 정답 구간만 학습하는 구성과 구별한다.

05 | Ubuntu 준비·설치

작업 폴더를 생성하고 VS Code에서 해당 폴더를 연다. 명령은 Ubuntu 터미널 기준이다.

python3.11 -m venv .venv # 프로젝트 전용 가상환경 생성 source .venv/bin/activate # 가상환경 활성화 python -m pip install --upgrade pip # 패키지 관리 도구 갱신 python -m pip install torch transformers datasets trl accelerate # 공식 학습 라이브러리 설치 python -c "import torch,trl; print(torch.**version**,trl.**version**,torch.cuda.is_available())" # 버전·GPU 확인

CUDA 지원 PyTorch는 드라이버·GPU에 맞춰 공식 설치 선택기를 사용한다. 설치 성공은 실습 PC에서 확인해야 한다. 전체 학습은 CUDA GPU를 권장하며, CPU만 있는 환경에서는 --dry-run을 사용한다.

06 | 전체 예제 코드·한글 주석

실습 파일 lesson08_sft.py는 첨부 원본을 그대로 사용한다. read_messages()는 JSONL의 messages/role/content를 검사한다. --dry-run은 모델 다운로드 없이 데이터를 검증한다.

SFTConfig는 max_length=256, max_steps=5, batch=1, gradient_accumulation_steps=2, eval_strategy="steps", eval_steps=5를 설정한다. SFTTrainer(model=..., args=..., train_dataset=..., eval_dataset=..., processing_class=...)로 학습기를 만든다.

실행: python lesson08_sft.py --train sample_processed/train.jsonl --validation sample_processed/validation.jsonl --dry-run

실제 학습: python lesson08_sft.py --train sample_processed/train.jsonl --validation sample_processed/validation.jsonl

실제 07차시 산출물 사용 시 --train processed/train.jsonl --validation processed/validation.jsonl로 경로를 변경한다.

07 | 코드 해설·검증 포인트

read_messages()는 파일 줄번호를 포함해 형식 오류를 보고한다. Dataset.from_list()는 파이썬 딕셔너리 목록을 Hugging Face Dataset으로 바꾼다. AutoTokenizer.from_pretrained()는 모델의 토큰화 규칙을 불러온다.

trainer.train()은 실제 가중치를 갱신한다. trainer.evaluate()는 검증 손실을 계산한다. trainer.save_model()은 전체 학습 모델을 저장한다. 이는 09차시의 LoRA 어댑터만 저장하는 방식과 다르다.

예상 출력 예: 데이터 확인: train=10, validation=2 / DRY RUN: 데이터 검증 완료, 모델 학습 미실행. 이 메시지는 동봉 예제에 대한 실제 검사 결과와 별도로 표시한다. 실제 GPU 학습 손실 수치는 실행 환경마다 달라지므로 임의의 성공 수치를 제시하지 않는다.

08 | 따라하기 60분 실습

① 07차시 processed 폴더를 복사하거나 동봉 sample_processed로 시작한다. ② 가상환경과 패키지 설치를 확인한다. ③ --dry-run을 실행해 데이터 건수를 확인한다. ④ CUDA GPU가 있으면 5스텝 학습을 수행한다. ⑤ 출력 폴더와 로그를 확인한다. ⑥ eval_loss와 train_loss를 기록한다.

검증표: 데이터 건수 / CUDA 사용 가능 여부 / 학습 5스텝 종료 / eval_loss 존재 / 모델 저장 경로 / 오류 여부.

이 예제는 소량의 샘플이므로 모델 성능을 평가할 때 06차시의 독립된 질문 세트를 추가로 사용한다. 같은 질문을 학습과 평가에 동시에 사용하지 않는다.

09 | 오류 대처·확장 실습

ModuleNotFoundError: 현재 가상환경을 확인하고 필요한 패키지를 설치한다. CUDA out of memory: max_length를 128로 줄이거나, 09차시 LoRA로 전환한다. GPU 없음: --dry-run까지만 수행한다.

TypeError: SFTConfig 인자 오류가 나면 설치된 trl 버전과 공식 문서의 버전을 비교한다. 구버전의 tokenizer= / max_seq_length=와 최신 processing_class= / max_length=를 혼용하지 않는다.

확장: max_steps 5→10, learning_rate 2e-5→1e-5를 한 변수씩 변경해 eval_loss를 비교한다. 데이터가 매우 작아 통계적 결론은 내리지 않는다.

10 | 평가·체크리스트·공식 출처

평가 질문: ① 학습 데이터와 검증 데이터를 분리하는 이유는? ② gradient_accumulation_steps=2는 무엇을 뜻하는가? ③ train_loss 감소만으로 품질 개선을 단정할 수 없는 이유는? ④ 08차시 전체 학습과 09차시 LoRA 학습의 저장 산출물 차이는?

체크리스트: [ ] JSONL 형식 검사 [ ] 데이터 누출 확인 [ ] 학습 라이브러리 버전 기록 [ ] GPU 상태 확인 [ ] 5스텝 로그 확보 [ ] 검증 손실 확인 [ ] 모델 저장 위치 확인.

공식 문서 확인일: 2026-11-04. TRL SFTTrainer: https://huggingface.co/docs/trl/sft_trainer ; Transformers 채팅 템플릿: https://huggingface.co/docs/transformers/main/chat_templating ; Datasets JSON: https://huggingface.co/docs/datasets/loading . 문서상 API 확인과 실제 설치 버전의 실행 검증은 별개다. 다음 09차시 SFT+LoRA로 연결한다.

전체 실습 코드 — lesson08_sft.py

```
"""08차시: 07차시 JSONL을 활용한 SFTTrainer 기초 학습 (전체 파라미터 학습)."""
import argparse
import json
from pathlib import Path


def read_messages(path):
    """대화 데이터의 역할과 내용 형식을 검사한다."""
    records = []
    for number, line in enumerate(Path(path).read_text(encoding="utf-8").splitlines(), 1):
        if not line.strip():
            continue
        row = json.loads(line)
        messages = row.get("messages")
        if not isinstance(messages, list) or not messages:
            raise ValueError(f"{path}:{number}: messages 목록 누락")
        for msg in messages:
            if msg.get("role") not in {"system", "user", "assistant"}:
                raise ValueError(f"{path}:{number}: role 오류")
            if not isinstance(msg.get("content"), str) or not msg["content"].strip():
                raise ValueError(f"{path}:{number}: content 오류")
        records.append({"messages": messages})
    if not records:
        raise ValueError(f"{path}: 유효한 데이터가 없습니다")
    return records


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--train", default="processed/train.jsonl")
    parser.add_argument("--validation", default="processed/validation.jsonl")
    parser.add_argument("--output", default="outputs/lesson08_sft")
    parser.add_argument("--dry-run", action="store_true", help="데이터만 검사하고 학습하지 않음")
    args = parser.parse_args()
    train_rows = read_messages(args.train)
    valid_rows = read_messages(args.validation)
    print(f"데이터 확인: train={len(train_rows)}, validation={len(valid_rows)}")
    if args.dry_run:
        print("DRY RUN: 데이터 검증 완료, 모델 학습 미실행")
        return

    # 학습 시에만 무거운 라이브러리를 불러온다.
    import torch
    from datasets import Dataset
    from transformers import AutoTokenizer
    from trl import SFTConfig, SFTTrainer

    model_id = "Qwen/Qwen2.5-0.5B-Instruct"  # 교육용 제작 선택
    tokenizer = AutoTokenizer.from_pretrained(model_id)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    if not torch.cuda.is_available():
        raise RuntimeError("이 실습의 전체 파라미터 학습은 CUDA GPU 환경에서 실행하세요")
    config = SFTConfig(
        output_dir=args.output,
        max_length=256,
        per_device_train_batch_size=1,
        gradient_accumulation_steps=2,
        max_steps=5,
        learning_rate=2e-5,
        logging_steps=1,
        eval_strategy="steps",
```

```
        eval_steps=5,
        save_strategy="no",
        report_to="none",
        fp16=True,
        seed=42,
        assistant_only_loss=False,  # 채팅 템플릿의 assistant mask 지원 여부와 무관하게 실행
    )
    trainer = SFTTrainer(
        model=model_id,
        args=config,
        train_dataset=Dataset.from_list(train_rows),
        eval_dataset=Dataset.from_list(valid_rows),
        processing_class=tokenizer,
    )
    result = trainer.train()
    metrics = trainer.evaluate()
    print("학습 손실:", result.training_loss)
    print("검증 지표:", metrics)
    trainer.save_model(args.output)
    tokenizer.save_pretrained(args.output)
    print("모델 저장:", args.output)


if __name__ == "__main__":
    main()
```

적대적 검증·개선 기록 (2026-11-05)

반영: 07차시의 실제 전처리 데이터(train=10, validation=2)를 실습 폴더에 복구했습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)